

Tablice statyczne wielowymiarowe

1. Tablice wielowymiarowe - podstawy

Tablice wielowymiarowe są konstruktem programistycznym, który (podobnie jak tablica jednowymiarowa) pozwala na agregację danych w zwartą grupę (elementy tego samego typu), co ułatwia zarządzanie tymi danymi.

Wielowymiarowość tablicy jest mechanizmem, który znacznie upraszcza zapis rozwiązania określonego problemu, poprzez łatwiejsze odwołanie się do „koordynatów” określonego elementu..

Przykłady tablic wielowymiarowych:

- arkusz kalkulacyjny – (tablica dwuwymiarowa) – MS Excel, LO Calc, • plansza do gry (tablica dwuwymiarowa) – szachy, sudoku, krzyżówka, • grafika – (tablica dwuwymiarowa) – piksele rozmieszczone w pionie i poziomie,
- film (surowe klatki) – (tablica trójwymiarowa) – jak wyżej plus kolejne obrazy następujące po sobie.
- Operacje macierzowe!

Możemy stworzyć tablicę o dowolnej liczbie wymiarów. Matematyka wszystko przyjmie, aczkolwiek powyżej 4 wymiarów, już sobie trudno wyobrazić „kształt” takiej tablicy.

2. Tablice dwuwymiarowe

Przykładowa deklaracja tablicy dwuwymiarowej:

```
int Tab[4][6];
```

`int` – przykładowy typ. Wszystkie elementy muszą być tego samego typu.

`Tab` – nazwa tablicy. Oczywiście dowolna.

`[]` - dwie pary nawiasów kwadratowych oznaczają, że jest to tablica dwuwymiarowa.

`[4][6]` - przykładowe rozmiary dla każdego z wymiarów. Rozmiar pierwszego wymiaru wynosi 4, rozmiar drugiego wymiaru wynosi 6.

Uwaga! Jako że tablica jest tworem dwuwymiarowym, wydawałoby się, że wygląda ona jak klasyczna tabelka "szerokość na wysokość". Nie ma wymogu jednoznacznego określenia gdzie jest zadeklarowana wysokość, a gdzie szerokość tablicy. To zależy tylko i wyłącznie od naszego własnego ustalenia.

Przykładowa tablica "tabliczka mnożenia", jaką chcemy

użyć:

0	0	0	0	0	0
0	1	2	3	4	5
0	2	4	6	8	10
0	3	6	9	12	15

Deklaracja:

```
int Tab[4][6];
```

Wypełnienie. Tablica dwuwymiarowa, a więc musimy użyć dwie pętle:

```
for(unsigned n = 0; n < 4; n++)
{
    for(unsigned k = 0; k < 6; k++)
    {
        Tab[n][k] = n * k;
    }
}
```

Można także zamienić pętle miejscami. Jednak należy pamiętać, aby indeksy były we właściwych miejscach. Inaczej ustawimy niewłaściwy fragment pamięci.

Sprawdzamy rozmiar danego wymiaru, jaka zmienna jest w pętli wykonującej się tyle razy co dany rozmiar wymiaru. Ta właśnie zmienna może być użyta jako indeks tego wymiaru.

0	0	0	0	0	0
0	1	2	3	4	5
0	2	4	6	8	10
0	3	6	9	12	15

Wyświetlenie. Tablica dwuwymiarowa, a więc musimy użyć dwie pętle.

```
// Pętla zewnętrzna steruje ile wyświetlono będzie rzędów for(unsigned y = 0; y < 4; y++)
{
    // Pętla wewnętrzna steruje ile będzie kolumn for(unsigned x = 0; x < 6; x++)
    {
        cout.width(4); // Wyrównanie wydruku
        cout << Tab[y][x];
    }
    // Po wydrukowaniu CAŁEJ LINII musimy dać "enter". // Inaczej wszystkie wydruki będą w jednej linii. cout <<
    "\n";
}
```

Podobnie jak wcześniej, **należy zawsze sprawdzić poprawność zmiennej użytej jako indeks w danym wymiarze.**

Gdyby pętle były wydrukowane na odwrót otrzymalibyśmy wynik jak poniżej. Oczywiście indeksy muszą być rozmieszczone prawidłowo w

wymiarach.

```
for( unsigned y = 0; y < 6; y++)
{
    for( unsigned x = 0; x < 4; x++)
    {
        cout.width(4); // Wyównanie wydruku
        cout << Tab[x][y];
    }

    cout << "\n";
}
```

0	0	0	0
0	1	2	3
0	2	4	6
0	3	6	9
0	4	8	12
0	5	10	15

Dobra zasada programistyczna. W sytuacjach, kiedy mamy pewność, który z wymiarów powinien być uznawany za szerokość, a który za wysokość (np. wydruk) zaleca się stosowanie indeksów/iteratorów „x”, „y”.

3. Tablice wielowymiarowe (trójwymiarowe)

Przykładowa deklaracja:

```
int Tab[3][4][5];
```

Ogólna zasada. Tablice wielowymiarowe konstruujemy poprzez użycie odpowiedniej liczby par nawiasów kwadratowych, wewnątrz opisując rozmiar kolejno każdego z wymiarów.

Podobnie jak w tablicach dwuwymiarowych, nie jest ściśle określona szerokość, wysokość, głębokość, itp. To zależy od naszej własnej interpretacji.

Tablica typu "tabliczka mnożenia", tyle, że trójwymiarowa.

```
// Wypełnienie. Tablica trójwymiarowa, musimy użyć trzech pętli. for( unsigned n = 0; n < 3; n++)
{
    for( unsigned k = 0; k < 4; k++)
    {
        for( unsigned m = 0; m < 5; m++)
        {
            Tab[n][k][m] = n * k * m;
        }
    }
}
```

Można także zamienić pętle miejscami. Jednak zawsze należy pamiętać, aby indeksy były we właściwych miejscach. Inaczej ustawimy niewłaściwy fragment pamięci.

Sprawdzamy rozmiar danego wymiaru, jaka zmienna jest w pętli

wykonującej się tyle razy co dany rozmiar wymiaru, ta właśnie zmienna może być użyta jako indeks tego wymiaru.

Wyświetlenie. Problemатyczne, bo trójwymiarowej tablicy nie bardzo da się wyświetlić na raz na ekranie. To co możemy zrobić, to "pociąć tablicę na plasterki", a następnie kolejno drukować te plasterki na ekranie.

```
// Tablica trójwymiarowa, a więc musimy użyć trzy pętle.  
// Pętla zewnętrzna, steruje ile będzie "plasterków"  
for( unsigned n = 0; n < 4; n++)  
{  
    // Pętla środkowa steruje ile będzie rzędów.  
    for( unsigned y = 0; y < 5; y++)  
    {  
        // Pętla wewnętrzna steruje ile będzie kolumn.  
        for( unsigned x = 0; x < 5; x++)  
        {  
            cout << 4; // Wyświetlenie wyniku  
            cout << Tab[ x ][ n ][ y ];  
        }  
  
        cout << "\n";  
    }  
  
    // Po wyświetleniu całego "plasterka" musimy dać "enter". // Inaczej wszystkie wyniki będą w jednej linii.  
    cout << "\n";  
}
```

Podobnie jak wcześniej, należy zawsze sprawdzić poprawność zmiennej użytej jako indeks w danym wymiarze.

4. Tablice wielowymiarowe – ułożenie w pamięci

Tablice wielowymiarowe, pomimo posiadania kilku wymiarów, są zadeklarowane w pamięci jako jeden ciągły blok.

Przykładowa deklaracja:

```
int Tab[ 3 ][ 5 ];
```

00	01	02	03	04
10	11	12	13	14
20	21	22	23	24

Kolejne komórki tablicy leżą w pamięci:

00	01	02	03	04	10	11	12	13	14	20	21	22	23	24
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Zatem po co tablica wielowymiarowa, skoro jednowymiarowa, także by dała radę? Chodzi o wygodę programisty. W zależności od problemu, o wiele lepiej zarządzać danymi, jeśli są one rozmieszczone „w przestrzeni” niż „na jednej ciągłej linii”. Rozmieszczenie danych w tablicach o małej liczbie wymiarów (dwa, trzy) można sobie z łatwością wyobrazić.

5. Zerowanie tablicy wielowymiarowej

Deklaracja jest tylko rezerwacją pamięci. Bity znajdujące się w tych komórkach pamięci, będą miały przełożenie na domyślne wartości komórek tablicy.

Aby wyzerować całą tablicę możemy zastosować zapis:

```
int Tab[3][5] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
```

Powyższy kod działa, ale niezbyt chętnie się zeruje tablicę w takiej postaci (co w przypadku tablicy o większej liczbie elementów?).

Lepiej wyzerować tablicę w sposób jak poniżej (działa niezależnie od ilości elementów ile byłoby użyte).

```
int Tab[3][5] = {0};
```

```
lib
```

```
int Tab[3][5] = 0;
```

6. Ustawianie wartości początkowych

Podobnie jak w przypadku tablicy jednowymiarowej, można także ustawiać wartości domyślne w tablicach wielowymiarowych:

```
int Tab[3][5] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15};
```

Powyższy zapis może się miejscami wydawać nieczytelny (nie widać, gdzie się wymiary przenikają). Zamiast niego można zastosować ustawianie z wykorzystaniem bloków (klamr), aby łatwiej było wizualnie zobaczyć jaką wartość, będzie przyporządkowana do jakiej komórki.

```
int Tab[3][5] = { {1, 2, 3, 4, 5},  
                  {6, 7, 8, 9, 10},  
                  {11, 12, 13, 14, 15} };
```

7. Ustawianie rozmiarów z użyciem stałych

Aby ułatwić sobie sprawę, możemy zadeklarować **stałe**, następnie

wykorzystać je do deklaracji lub jako ograniczników wykonania pętli.

```
const unsigned Wysokosc = 6;
```

```
const unsigned Szerokosc = 8;
```

```
int Tab[ Wysokosc ][ Szerokosc ];
```

```
// W pętliach używamy Szerokosc i Wysokosc  
// jako ograniczników indeksów
```

```
for( unsigned y = 0; y < Wysokosc; y++ )  
{  
    for( unsigned x = 0; x < Szerokosc; x++ )  
    {  
        Tab[ y ][ x ] = y * x;  
    }  
}
```